

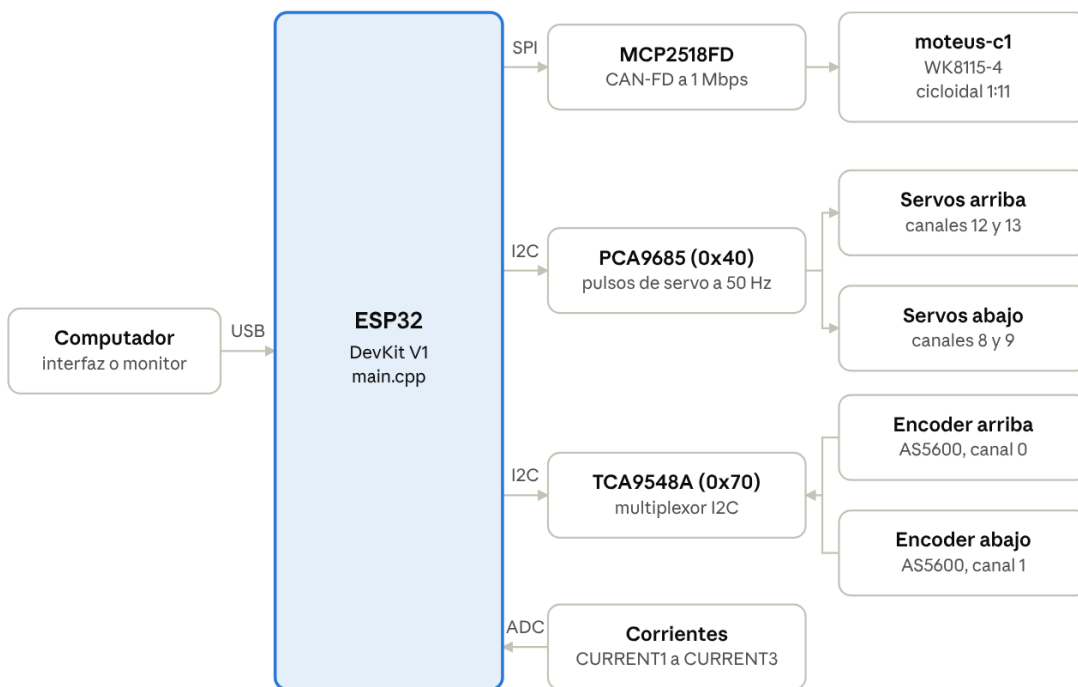
Guía de Uso: Firmware e Interfaz Banco de Pata

Leonardo Monsalve Gómez

Cómo está armado el sistema

Un ESP32 controla tres partes que funcionan al mismo tiempo y por separado: la pata con el brushless (por CAN), dos pares de servos (por el PCA9685) y dos encoders (por el TCA9548A). El computador manda los comandos por USB, desde el monitor serie o desde la interfaz.

El ESP32 controla el brushless por CAN y los servos y encoders por I2C



arquitectura del sistema · ESP32 y sus tres partes

Cada parte responde solo a su comando, y si una no inicia (CAN o I2C), las otras siguen funcionando.

Archivo	Qué es
platformio.ini	Configuración de PlatformIO con dos entornos: esp32dev y diag_pca
src/main.cpp	Programa principal del ESP32: brushless, servos, encoders y telemetría
src/diagnostico/diag_pca9685.cpp	Diagnóstico del PCA9685 y de los servos

Archivo	Qué es
interfaz_python/banco_pata.py	Interfaz gráfica (Python)
interfaz_python/requirements.txt	Librerías de la interfaz

Instalación en Windows

En Windows se necesitan tres cosas: PlatformIO para cargar el ESP32, el driver CH340 para ver el puerto COM y Python con las librerías de la interfaz.

1. **VS Code y PlatformIO:** instala VS Code y, en Extensiones, busca e instala **PlatformIO IDE**. Abre la carpeta del proyecto (la que tiene `platformio.ini`).
2. **Driver del ESP32 (CH340):** conecta el ESP32 y abre el Administrador de dispositivos, en Puertos (COM y LPT). Si sale "USB-SERIAL CH340 (COM5)", anota el número. Si sale con advertencia, instala el driver CH341SER de WCH y vuelve a conectar.
3. **Python:** instala Python 3.12 desde python.org y marca **Add python.exe to PATH**. Python 3.14 puede no tener todavía la librería PySide6.
4. **Librerías de la interfaz:** abre PowerShell en la carpeta `interfaz_python` y ejecuta:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Si `Activate.ps1` da error de permisos, ejecuta una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` y repite.

5. **Intérprete de VS Code:** para que el botón  use ese Python, presiona `Ctrl+Shift+P`, escribe **Python: Select Interpreter** y elige el del `venv` (o Python 3.12). Si sale `No module named 'pyqtgraph'`, el botón está usando otro Python sin las librerías.

En `platformio.ini` deja comentadas (con `;`) las líneas `upload_port` y `monitor_port`: PlatformIO encuentra el COM solo.

Instalación en Linux (Ubuntu)

En Linux el puerto es `/dev/ttyUSB0` y hay que dar permisos al usuario; el código es el mismo que en Windows.

1. **PlatformIO:** igual que en Windows (VS Code + PlatformIO IDE), o por terminal con `pio`.
2. **Quitar brltty:** este programa toma el puerto del CH340 y el ESP32 nunca aparece como `/dev/ttyUSB0`.

```
sudo apt remove brltty
```

Desconecta y vuelve a conectar el ESP32 y revisa con `ls /dev/ttyUSB*`.

3. **Permiso del puerto:** agrega tu usuario al grupo `dialout`, cierra sesión y vuelve a entrar.

```
sudo usermod -aG dialout $USER
```

4. **Librerías de la interfaz:** en la carpeta `interfaz_python`:

```
sudo apt install python3-venv libxcb-cursor0
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Sin `libxcb-cursor0` la ventana no abre y sale "Could not load the Qt platform plugin xcb".

Al copiar el proyecto desde Windows, **no copies la carpeta venv**: un venv de Windows no sirve en Linux y se crea uno nuevo.

Cargar cada programa en el ESP32

El proyecto tiene dos programas independientes; el ESP32 corre uno a la vez y cargar uno reemplaza al otro.

Entorno	Archivo	Para qué
esp32dev	src/main.cpp	Programa completo: brushless, servos y encoders
diag_pca	src/diagnostico/diag_pca9685.cpp	Probar solo el PCA9685 y los servos

Desde VS Code: en la barra azul de abajo, haz clic en el entorno (`env:esp32dev` o `env:diag_pca`), luego **Upload** (→) y **Monitor** (🔌). Si cambiaste librerías o algo falla raro, primero **Clean** (PlatformIO → Project Tasks → entorno → General → Clean).

Desde la terminal (Windows y Linux):

```
pio run -e esp32dev -t upload -t monitor
pio run -e diag_pca -t upload -t monitor
pio run -t clean
```

Antes de cada Upload cierra el monitor serie y la interfaz: solo un programa puede usar el puerto. Si sale "Acceso denegado" o "PermissionError", hay otro programa con el puerto abierto.

Diagnóstico del PCA9685 (diag_pca)

El diagnóstico prueba solo los servos, sin brushless ni encoders, escribiendo directo en los registros del chip. Úsalo cuando un servo no responda.

Al arrancar, el diagnóstico escanea el bus I2C, explica los registros, prueba la comunicación y configura el chip a 50 Hz. Debe decir `0x40 PCA9685, Comunicacion OK y PCA9685 configurado ... 50 Hz`.

Comando	Qué hace
pa0 / pb0	Par de arriba (canales 12 y 13) o de abajo (8 y 9) al centro
pa30, pa-30	Mover el par de arriba; los dos servos al mismo ángulo, de -90 a 90
c12 y g45	Elegir un canal y moverlo solo

Comando	Qué hace
1500	Pulso directo en microsegundos al canal elegido (sin movimiento suave)
v10	Velocidad de los movimientos suaves en °/s (por defecto 20)
b	Barrido lento del canal elegido
p	Probar los 16 canales, uno por segundo
m	Medir el pulso real: une el pin S del canal al GPIO 4
t	Ver en qué canales del TCA9548A hay un AS5600
e, r, k, i	Escaneo I2C, registros, prueba de comunicación, configurar a 50 Hz
x	Apagar todo (servos sin fuerza)

Cómo leer el resultado: si nadie responde en el escaneo, revisa SDA en 21, SCL en 22 y el VCC del módulo. Si responde pero los servos no se mueven, revisa la fuente en V+, el GND común y el pin OE (a GND). Si la fuente suena o marca CC, envía x: algo está trabado.

El diagnóstico recuerda la última posición de cada canal aunque se apague el ESP32, así que todos los movimientos salen suaves salvo el primerísimo comando de cada canal.

Programa principal (esp32dev)

Cada comando mueve solo su parte: un ángulo sin letra es para el brushless, a para los servos de arriba y b para los de abajo. Se escriben en el monitor serie o en la caja de la consola de la interfaz.

Parte	Comando	Qué hace
Brushless	-90 a 90	Ir a ese ángulo (0 = horizontal, 90 = vertical arriba, -90 = colgando)
Brushless	s	Stop: motor libre, la pata cae (sostenerla)
Brushless	v	Confirmar el cero al arrancar (pata colgando, coincide con el transportador)
Brushless	z / h	Guardar el cero con la pata colgando (queda como -90) / el sentido con la pata horizontal

Parte	Comando	Qué hace
		(0), después de s
Brushless	m	Medir el par de gravedad estando en 0 (horizontal)
Brushless	t30	Límite de par en N·m (hasta 40)
Brushless	c / r	Ver / borrar la calibración
Servos arriba	a0, a45, a-45	Ángulo de -90 a 90, 0 = centro
Servos abajo	b0, b30, b-30	Igual, para el par de abajo
Servos	ax / bx	Soltar ese par (sin fuerza)
Servos	az / bz	Alinear el encoder con el par quieto
Servos	av10 / bv10	Velocidad del par en °/s (por defecto 20)
General	e	Estado de todo
General	x	PARAR TODO: brushless libre y servos sueltos
General	i	Cero de los sensores de corriente (sin carga)
General	p1 / p0	Telemetría para la interfaz (la interfaz la activa sola)

Al arrancar imprime el estado de cada parte: PCA9685: ... OK, el estado de cada encoder y MCP2518FD OK: brushless disponible. Si una parte da ERROR, las otras siguen funcionando.

El cero del brushless se verifica en cada arranque: hasta enviar v (o z), el brushless no se mueve y responde Verifica el cero primero (v o z).

Interfaz gráfica (banco_pata.py)

La interfaz abre en **SIMULACIÓN** (etiqueta naranja) para practicar; al conectar el ESP32 muestra y controla el hardware real.

Abrir: activa el venv y ejecuta `python banco_pata.py` (Windows) o `python3 banco_pata.py` (Linux). También se puede pasar el puerto: `python banco_pata.py COM5`.

1. Conectar

1. Cierra el monitor de PlatformIO.

2. Arriba a la derecha elige el puerto (COM5 o `/dev/ttyUSB0`; ⌵ si no aparece) y pulsa **Conectar ESP32**.
3. El ESP32 se reinicia. En la consola deben salir los mensajes de arranque y arriba • **ESP32 en vivo**
• 25 Hz.

2. Calibrar el brushless (panel MANDO)

1. Si ya estaba calibrado: con la pata colgando quieta y el ángulo cerca de -90, pulsa **Confirmar cero [v]**.
2. Primera vez o cero corrido: **STOP**, pata colgando → **Cero colgando [z]**; pata horizontal a mano → **Sentido [h]**.
3. Deben quedar en verde ✓ Cero guardado, ✓ Sentido y ✓ Cero verificado.
4. Opcional: botón **0 (horizontal)**, esperar quieta y **Gravedad [m]**.

3. Calibrar los servos (tarjetas SERVOS ARRIBA y ABAJO)

1. Pulsa **0** en las dos tarjetas (la primera vez van directo; después siempre suave).
2. Prueba **+45** y **-45** y abre el rango de a poco; si algo choca o la fuente suena, **PARAR TODO [x]**.
3. Con cada par quieto en 0 y el imán en **OK**, pulsa **Alinear [az]** y **Alinear [bz]**.
4. Ajusta la velocidad con el número de °/s y **Vel**.

4. Mover todo

Qué	Dónde
Pata (brushless)	MANDO: barra -90 a 90 + Ir , o botones -90 a 90 (0 = horizontal, 90 = arriba)
Servos de arriba	Tarjeta SERVOS ARRIBA: barra -90 a 90 + Ir , o -90, -45, 0, +45, +90
Servos de abajo	Tarjeta SERVOS ABAJO, igual
Detener	STOP (solo brushless), Soltar [ax]/[bx] (un par), PARAR TODO [x] (todo)

Las tres partes se pueden mover al mismo tiempo; cada una avanza a su propia velocidad.

5. Qué vigilar

- **Batería:** elige tu batería en el selector (o Fuente de banco); bajo 20 % se pone en rojo.
- **Temperatura de la placa:** amarillo desde 50 °C (el moteus reduce corriente), rojo desde 75 °C.
- **Fault:** si aparece, la interfaz dice qué significa y qué revisar (la 33 fue un falso contacto en una fase).
- **Imán de cada encoder:** debe decir OK; "no detectado" o "débil" = imán ausente o lejos del AS5600.
- **Gráficas:** ángulo de la pata, par y servos (línea continua = encoder, rayas = comando).

Configuración que se puede cambiar en main.cpp

Todo lo ajustable está al inicio de main.cpp, en la parte CONFIGURACIÓN; después de cambiar algo hay que volver a cargar con Upload.

```
ConfigPar CFG[2] = {  
  // nombre  letra canales trims  min  max  vel  enc signo  
  {"Arriba",  'a', 12, 13,  0, 0, -90, 90, 20,  0,  1},  
  {"Abajo",   'b',  8,  9,  0, 0, -90, 90, 20,  1,  1},  
};
```

Campo	Qué cambia
canales	Salidas del PCA9685 de los dos servos del par (los dos reciben el mismo ángulo)
trims	Ajuste fino en μ s de cada servo para alinearlos (unos 10 μ s por grado)
min / max	Rango seguro del par en grados; fuera de ese rango el comando se rechaza
vel	Velocidad por defecto en $^{\circ}/s$
enc	Canal del TCA9548A del encoder de ese par
signo	+1 o -1 para que el encoder crezca igual que el servo

Constante	Valor	Para qué
ACEL_SERVOS	40	Aceleración de los servos en $^{\circ}/s^2$ (menor = más suave)
SERVO_MIN_US / SERVO_MAX_US	500 / 2500	Pulso para -90 y +90
PCA_OSC_HZ	27000000	Oscilador del PCA9685 (ajustar si m del diagnóstico lo pide)
SENS_V_POR_A	0.100	Sensibilidad del sensor de corriente (ACS712 de 20 A; 5 A = 0.185; 30 A = 0.066)
VEL_LIMITE / ACEL_LIMITE	0.15 / 0.3	Movimiento del brushless en vueltas/s y vueltas/s ² (igual que en tvew)

La calibración del moteus (kp 200, kd 4, ki 50, ilimit 12, 6 A) está guardada en el propio moteus desde tvew; el ESP32 no la cambia.

Solución de problemas

Estos son los problemas que ya aparecieron en las pruebas y cómo se resolvieron.

Síntoma	Causa	Solución
"Acceso denegado" / PermissionError en COM5	Otro programa tiene el puerto (monitor, interfaz, Arduino IDE)	Cerrarlo; si sigue, desconectar y reconectar el USB
Upload va a /dev/ttyS0 (Linux)	brltty tomó el CH340	sudo apt remove brltty y reconectar
SPI.h: No such file or directory	Librería vieja de Adafruit	Usar el platformio.ini actual y hacer Clean
No module named 'pyqtgraph'	Las librerías no están en el Python que corre	Instalar en ese Python o elegir el venv como intérprete
La interfaz dice "Sin datos"	El ESP32 no tiene el firmware con telemetría	Cargar el main.cpp actual
La fuente suena como en corto	Servos trabados o peleándose	x de inmediato; revisar horns y mecánica
Servos dan un tirón pero no se mueven	Fuente limitando corriente	Subir el límite a 2,5 o 3 A por servo; GND común
Los servos de arriba se mueven con b	Canales intercambiados	Arriba = 12 y 13, abajo = 8 y 9 en la tabla CFG
Encoder "no detectado" o "débil"	Imán ausente o lejos	Centrar el imán a 0,5 a 3 mm del AS5600
El brushless no se mueve	Cero sin verificar o CAN	Pulsar Confirmar cero [v] ; si la consola da error del MCP2518FD, revisar SPI y 3V3; desconectar el mjcanfd del bus
Fault 33	Falso contacto en una fase	Revisar las tres fases del motor
Fault 102	Límite de par del comando	Subir con t30
El primer movimiento de un servo salta	No hay posición guardada todavía	Poner el brazo a mano en el centro y enviar a0 y b0 una vez